SAMSUNG SW AI ACADEMY FOR YOUTH

함께가요 미래로!
Enabling People

SSAFY

Django

ORM

Python



SAMSUNG SW AI ACADEMY FOR YOUTH

함께가요 미래로!
Enabling People

목 차

ORM

QuerySet API

QuerySet API 실습

- Create
- Read
- Update
- Delete

ORM with view

- 전체 게시글 조회

참고

- Field lookups
- ORM, QuerySet API를 사용하는 이유

학습 시작

Django는 Python 문법만으로 데이터베이스와 대화할 수 있다.

- Django **ORM** 첫걸음

1. Django **ORM**은 파이썬과 데이터베이스 사이의 **통역사 역할**을 하며, 선언적 코드만으로 데이터 저장·조회 명령을 실행한다.
2. **QuerySet API**로 게시글을 **생성·조회·수정·삭제**하면서 데이터베이스 명령어를 직접 작성하지 않아도 된다.

- Python 클래스 → DB 레코드 전환 과정을 이해하고, 객체 지향 코드만으로 데이터를 생성·조회·수정·삭제할 수 있습니다.

학습 목표

- ✓ ORM의 개념과 데이터베이스-파이썬 간 역할을 설명할 수 있다.
- ✓ QuerySet API 기본 구조를 이해하고, CRUD 전 과정을 파이썬 코드로 수행할 수 있다.
- ✓ Field lookup의 개념과 사용 방법을 이해할 수 있다
- ✓ Django Shell에서 모델 인스턴스를 만들고 save()로 저장해 객체 → 레코드 변환 흐름을 실습할 수 있다.
- ✓ View 함수에서 QuerySet을 가져와 템플릿에 전달하는 Model → View → Template 데이터 흐름을 구현할 수 있다.

ORM

ORM이란?

매핑(Mapping)
서로 다른 형태를 가진 코드 상의 '객체'와 데이터베이스의 '테이블(데이터)'이 서로 1:1로 대응되도록 연결해 주는 것

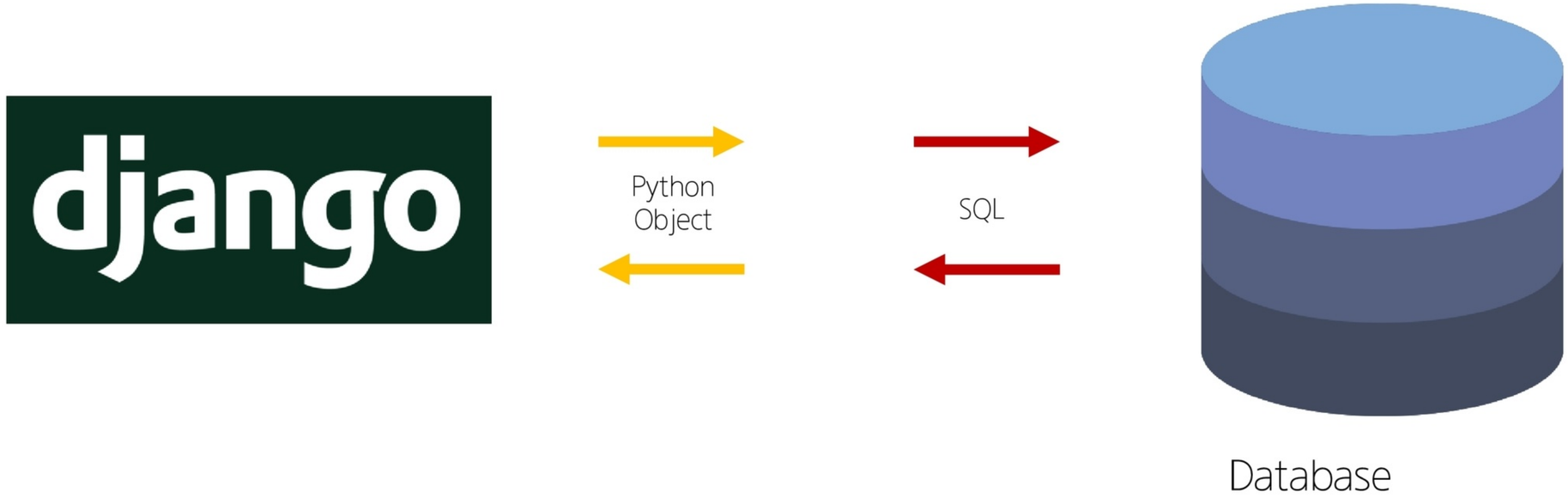
<개념3_Django_ORM_매핑>

ORM

Object-Relational Mapping
객체 지향 프로그래밍 언어의 객체(Object)와
데이터베이스의 데이터를 매핑(Mapping)하는 기술

문제 상황: 언어 차이로 인한 소통 불가

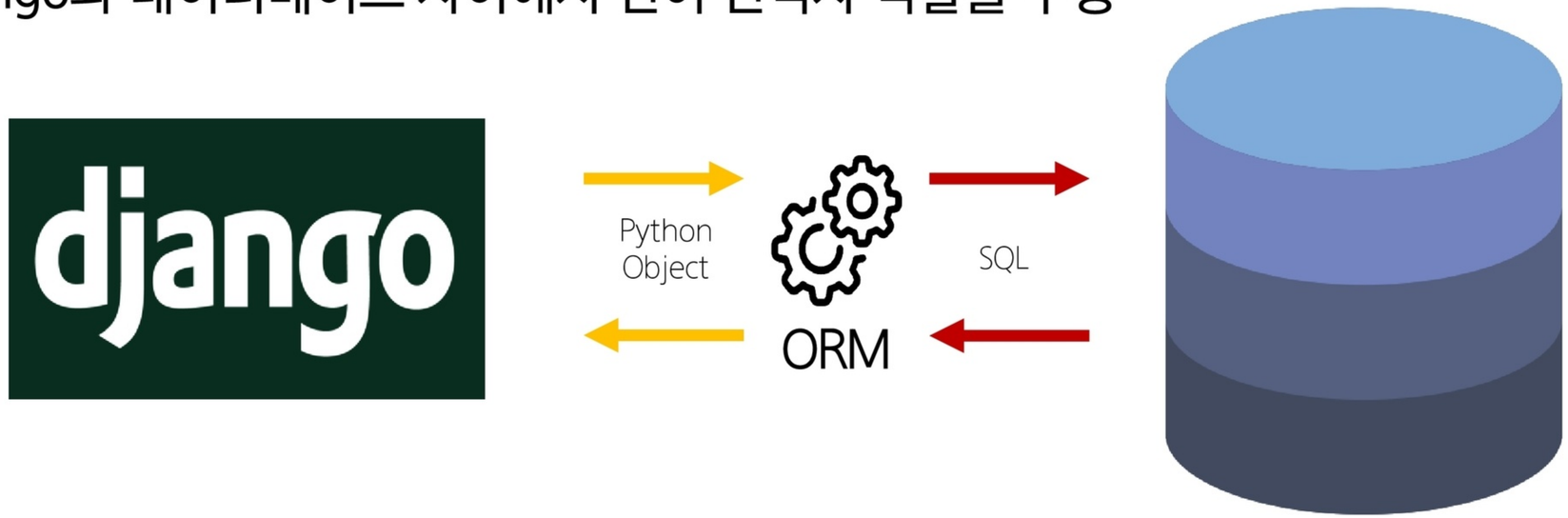
- Django는 Python 언어를 사용하지만 데이터베이스는 SQL 언어를 사용함



<그림1_Django_ORM_ORM의 역할 소통 불가>

ORM의 역할: 번역자 역할

- ORM은 Django와 데이터베이스 사이에서 언어 번역자 역할을 수행



<그림2_Django_ORM_ORM의 역할 소통 가능 with ORM>

TIP

- 마치 외국인과 대화할 때 '통역사'가 중간에서 **언어를 번역**해 주는 것과 같습니다.
- 덕분에 개발자는 **SQL을 직접 작성하지 않고도** 파이썬 코드로 데이터를 다룰 수 있습니다.
- 코드 작성 시간을 줄이고, 실수를 줄이는 데 큰 도움을 줍니다.

Django의 데이터 상호작용: ORM이 일하는 방법

- ORM은 Django 개발자를 위해 'QuerySet API'라는 특별한 도구를 제공함
  - QuerySet API는 ORM의 기능을 개발자가 Python 코드 안에서 객체 지향적이고 직관적인 방식으로 데이터베이스를 조작할 수 있도록 제공하는 **인터페이스**

QuerySet API

QuerySet API란?

QuerySet API

데이터베이스의 복잡한 SQL 쿼리문을,

SQL 대신 직관적인 Python 코드로 데이터베이스에 요청할 수 있게 해주는 도구(인터페이스)

- 개발자는 SQL을 직접 작성하지 않고도, `.filter()`, `.exclude()`, `.order_by()` 등 파이썬다운 메서드를 사용하여 원하는 데이터를 손쉽게 생성, 조회, 수정, 삭제할 수 있습니다.
- 코드의 가독성을 높이고, 개발 생산성을 극대화하는 Django ORM의 핵심 기능입니다.

Query란?

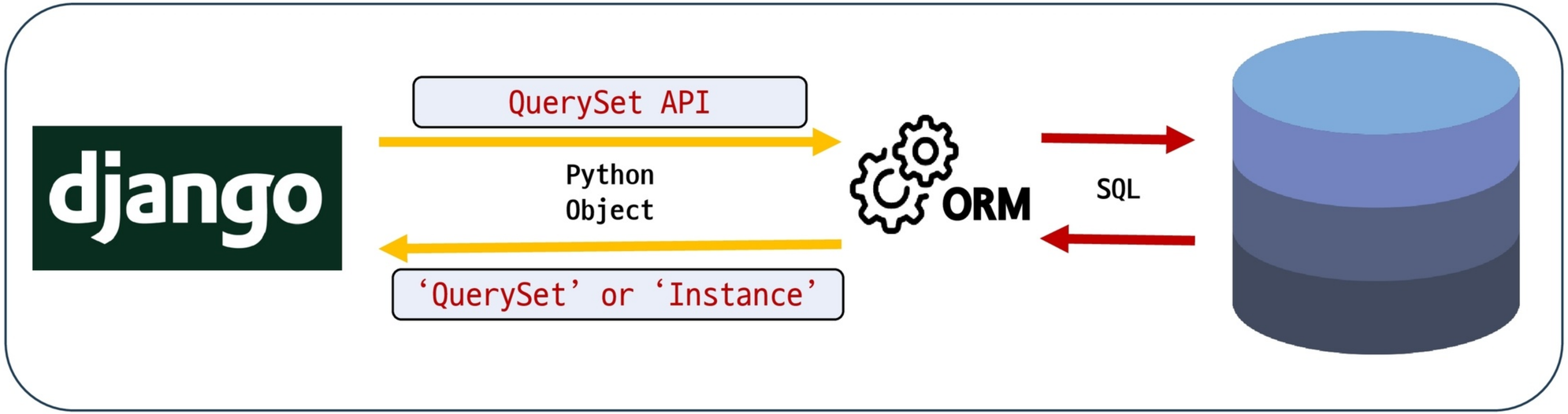
- 데이터베이스에 특정한 데이터를 보여 달라는 요청
- “쿼리문을 작성한다.”
  - “원하는 **데이터**를 얻기 위해 데이터베이스에 요청을 보낼 **코드**를 작성한다.”
- Django에서 Query가 처리되는 과정 정리
  1. 파이썬 코드 → ORM: 개발자의 QuerySet API(파이썬 코드)가 ORM으로 전달
  2. ORM → SQL 변환: ORM이 이를 데이터베이스용 SQL 쿼리로 변환하여 데이터베이스에 전달
  3. DB 응답 → ORM: 데이터베이스가 SQL 쿼리를 처리하고 결과 데이터를 ORM에 반환
  4. ORM → QuerySet 변환: ORM이 데이터베이스의 결과를 QuerySet (파이썬 객체) 형태로 변환하여 우리에게 전달

QuerySet이란?

- Django ORM을 통해 만들어진 자료형
- 데이터베이스에서 전달받은 **객체 목록(데이터 모음)**
- 순회 가능한 데이터로 **0개 이상** 데이터를 불러와 사용 가능함
- 단, get()처럼 **단일 객체를 조회하는 메서드**는 QuerySet이 아닌 모델의 인스턴스를 반환함

QuerySet API와 ORM의 동작 방식

1. Django → DB: Django(QuerySet API)에서 ORM을 통해 데이터베이스로 정보를 요청할 때
   - SQL 쿼리로 변환되어 데이터베이스로 전달됨
2. DB → Django: 데이터베이스가 요청에 대한 응답을 보낼 때,
   - ORM은 이 SQL 결과를 다시 파이썬이 이해할 수 있는 Python Object
   - QuerySet 또는 Instance 형태로 변환하여 Django로 반환



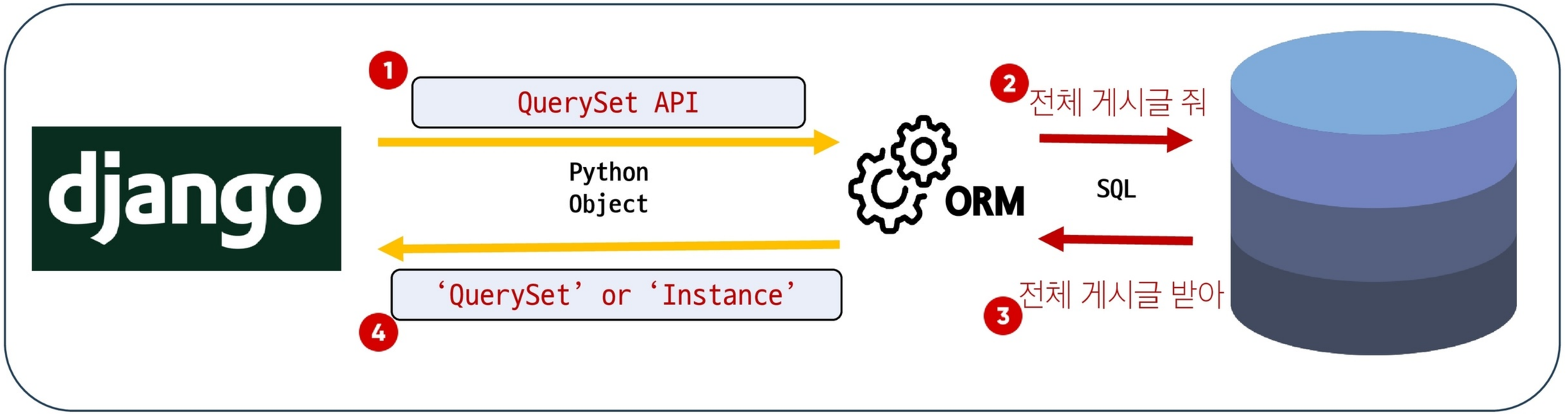
<그림3_Django_ORM_ORM의 역할 소통 가능 with ORM with querySet Api>

QuerySet API 구문 기본 구조



- `Article` (모델 클래스)
  - 역할: 데이터베이스 테이블에 대한 Python 클래스 표현
  - `articles_article` 테이블의 스키마(필드, 데이터 타입 등)를 정의하며, Django ORM이 데이터베이스와 상호작용할 때 사용하는 기본적인 구조체
- `.objects` (매니저, manager)
  - 역할: 데이터베이스 질의(Query, 질문과 요청) 작업을 위한 기본 인터페이스
  - 모델 클래스가 데이터베이스 쿼리 작업을 수행할 수 있도록 하는 진입점
  - Django는 모든 모델에 objects라는 이름의 매니저를 자동으로 추가하며, 이 매니저를 통해 `.all()`, `.filter()` 등의 쿼리 메서드를 호출
- `.all()` (QuerySet API 메서드)
  - 역할: 특정 데이터베이스 작업을 수행하는 명령
  - 매니저를 통해 호출되는 메서드로, 해당 모델과 연결된 테이블의 모든 레코드(rows)를 조회하라는 SQL 쿼리를 생성하고 실행

QuerySet API와 ORM의 동작 방식 예시



<그림4_Django_ORM_ORM의 역할 소통 가능 with ORM with querySet Api with examples>

QuerySet API 실습

CRUD란?

CRUD

대부분의 소프트웨어가 가지는 기본적인 데이터 처리 기능인 생성, 조회, 수정, 삭제를 묶어 이르는 말

Django에서는 QuerySet API를 통해, 복잡한 SQL문 없이 Python 코드로 이러한 CRUD 작업을 직관적으로 수행할 수 있음

- Create: 새로운 데이터를 생성 (저장)
- Read: 기존 데이터를 조회
- Update: 기존 데이터를 수정 (갱신)
- Delete: 기존 데이터를 삭제

Create

QuerySet API 실습 사전 준비

- 외부 라이브러리 설치 및 의존성 기록
  - **IPython**은 일반 파이썬 셸(명령창)보다 **자동 완성 등** 편리한 파이썬 **작업 환경**을 만들어주는 도구

```
$ pip install ipython
```

```
$ pip freeze > requirements.txt
```

QuerySet API 실습 사전 준비

- Django Shell 접속하기: Django Shell이란?
  - Django 프로젝트의 코드를 명령창에서 바로 실행하고 테스트하는 특별한 파이썬 환경
  - Django 환경 내에서 실행되기 때문에 입력하는 QuerySet API 구문이 실제 데이터베이스(db.sqlite3)에 그대로 반영됨
- Django Shell 접속하기

```
$ python manage.py shell
```

```
$ python manage.py shell
7 objects imported automatically (use -v 2 for details).

Python 3.11.9 (tags/v3.11.9:de54cf5, Apr  2 2024, 10:12:12) [MSC v.1938 64 bit (AMD64)]
Type 'copyright', 'credits' or 'license' for more information
IPython 9.4.0 -- An enhanced Interactive Python. Type '?' for help.
Tip: Use `object?` to see the help on `object`, `object??` to view its source

In [1]:
```

<그림5_Django_ORM_django shell>

QuerySet API 실습 사전 준비

- Shell "-v" 옵션 (기본값: 1)
  - 출력 상세도(verbosity level) 설정: 일반적인 정보 외에 더 많은 디버깅 정보나 진행 상황 메시지를 보여달라는 요청
  - 아래 예시는 shell 시작 시 **Django 프로젝트에 등록된 model**이 자동으로 import 된 내용이 출력된 것

```
$ python manage.py shell -v 2
```

```
$ python manage.py shell -v 2
7 objects imported automatically:

  from django.contrib.sessions.models import Session
  from django.contrib.contenttypes.models import ContentType
  from django.contrib.auth.models import User, Group, Permission
  from django.contrib.admin.models import LogEntry
  from articles.models import Article
```

<그림6_Django_ORM_django shell -v2>

이론

데이터 객체를 만드는(생성하는) 3가지 방법

1. 빈 객체 생성 후 값 할당 및 저장
2. 초기 값과 함께 객체 생성 및 저장
3. create() 메서드로 한 번에 생성 및 저장

첫번째 방법: 빈 객체 생성 후 값 할당 및 저장(1/3)

```
# 특정 테이블에 새로운 행을 추가하여 데이터 추가

>>> article = Article() # Article(class)로부터 article(instance) 생성
>>> article
<Article: Article object (None)>

>>> article.title = 'first' # 인스턴스 변수(title)에 값을 할당
>>> article.content = 'django!' # 인스턴스 변수(content)에 값을 할당
```

```
# save를 하지 않으면 아직 DB에 값이 저장되지 않음
>>> article
<Article: Article object (None)>
>>> Article.objects.all()
<QuerySet []>
```

첫번째 방법: 빈 객체 생성 후 값 할당 및 저장(2/3)

```
# save를 호출하고 확인하면 저장된 것을 확인
>>> article.save()
>>> article
<Article: Article object (1)>

>>> article.id
1

>>> article.pk
1

>>> Article.objects.all()
<QuerySet [<Article: Article object (1)>]>
```

첫번째 방법: 빈 객체 생성 후 값 할당 및 저장(3/3)

```
# 인스턴스 article을 활용하여 인스턴스 변수 활용하기
>>> article.title
'first'

>>> article.content
'django!'

>>> article.created_at
datetime.datetime(2026, 12, 31, 6, 55, 42, 322526, tzinfo=datetime.timezone.utc)
```

두번째 방법: 초기 값과 함께 객체 생성 및 저장

```
>>> article = Article(title='second', content='django!')
# 아직 저장 되어있지 않음
>>> article
<Article: Article object (None)>

# save를 호출해야 저장됨
>>> article.save()

>>> article
<Article: Article object (2)>

>>> Article.objects.all()
<QuerySet [<Article: Article object (1)>, <Article: Article object (2)>]>
```

```
# 값 확인
>>> article.pk
2

>>> article.title
'second'

>>> article.content
'django!'
```

- 첫번째 방법과 두번째 방법 모두 결국 **save 메서드**를 호출해야 비로소 **DB에 데이터**가 저장됨
- 테이블에 한 행(레코드)이 쓰여진 것

이론

save() 메서드란?

save()

객체를 데이터베이스에 **저장**하는 인스턴스 메서드

TIP

save() 가 필요한 경우는?

- 객체를 만든 뒤 저장 전에 값을 더 채우거나 바꿔야 할 때 생성과 저장을 나눠서 save()를 호출합니다.

❖ save()는 입력값이 올바른지 검사하지 않고 그대로 저장합니다. 입력값 검사는 이후 Form에서 배웁니다.

세번째 방법: create() 메서드로 한 번에 생성 및 저장

```
# 위 2가지 방법과 달리 생성과 동시에 저장되고, 생성된 객체가 반환된다.

>>> Article.objects.create(title='third', content='django!')
<Article: Article object (3)>
```

TIP save()를 명시적으로 호출하지 않는 것처럼 보이는 이유는 create() 메서드 자체가 객체 생성과 데이터베이스 저장을 한 번에 처리하는 단축 메서드이기 때문입니다.

Read

이론

대표적인 조회 메서드

- QuerySet 반환 메서드
  - `all()`
  - `filter()`
- QuerySet을 반환하지 않는 메서드
  - `get()`



QuerySet
데이터베이스에서 전달받은 객체 목록

<개념1_Django_ORM_쿼리셋>

QuerySet 반환 메서드: all()

all()

전체 데이터 조회

```
>>> Article.objects.all()
<QuerySet [<Article: Article object (1)>, <Article: Article object (2)>, <Article: Article object (3)>]>
```

QuerySet 반환 메서드: filter()

filter()

주어진 매개변수와 일치하는 객체를 포함하는 QuerySet 반환

```
>>> Article.objects.filter(content='django!')
<QuerySet [<Article: Article object (1)>, <Article: Article object (2)>, <Article: Article object (3)>]>

>>> Article.objects.filter(title='abc')
<QuerySet []>

>>> Article.objects.filter(title='first')
<QuerySet [<Article: Article object (1)>]>
```

QuerySet을 반환하지 않는 메서드: get()

get()

주어진 매개변수와 일치하는 **객체를 반환**

```
>>> Article.objects.get(pk=1)
<Article: Article object (1)>

>>> Article.objects.get(pk=100)
DoesNotExist: Article matching query does not exist.

>>> Article.objects.get(content='django!')
MultipleObjectsReturned: get() returned more than one Article -- it returned 3!
```

이론

get()의 특징

- 객체를 찾을 수 없으면 DoesNotExist 예외를 발생시키고,
  둘 이상의 객체를 찾으면 MultipleObjectsReturned 예외를 발생시킴
- 위와 같은 특징을 가지고 있기 때문에 primary key와 같이 고유성(uniqueness)을 보장하는 조회에서 사용해야 함

Primary key(pk)
DB 테이블에서 각 행을 고유하게 식별할 수 있는 속성

<개념2_Django_ORM_프라이머리 키>

```
>>> Article.objects.get(pk=1)
<Article: Article object (1)>

>>> Article.objects.get(pk=100)
DoesNotExist: Article matching query does not exist.

>>> Article.objects.get(content='django!')
MultipleObjectsReturned: get() returned more than one Article -- it returned 3!
```

Update

데이터 수정 방법

- 인스턴스 변수를 변경 후 save 메서드 호출

```
# 수정할 인스턴스 조회
>>> article = Article.objects.get(pk=1)

# 인스턴스 변수를 변경
>>> article.title = 'byebye'

# 저장
>>> article.save()
```

TIP

- save()는 pk가 없으면 새로 생성(INSERT), pk가 있으면 수정(UPDATE)합니다.
- 그래서 Create와 Update 모두 save()를 사용합니다.

Delete

데이터 삭제 방법

- 삭제하려는 데이터 조회 후 delete 메서드 호출

```
# 삭제할 인스턴스 조회
>>> article = Article.objects.get(pk=1)

# delete 메서드 호출 (삭제된 개수와 모델별 삭제 개수가 튜플로 반환)
>>> article.delete()
(1, {'articles.Article': 1})

# 삭제한 데이터는 더이상 조회할 수 없음
>>> Article.objects.get(pk=1)
DoesNotExist: Article matching query does not exist.
```

※ 1번 글을 지운 뒤 새 글을 만들면 1번이 아니라 4번이 됩니다. 한 번 쓴 pk는 다시 쓰지 않습니다.

SAMSUNG SW AI ACADEMY FOR YOUTH

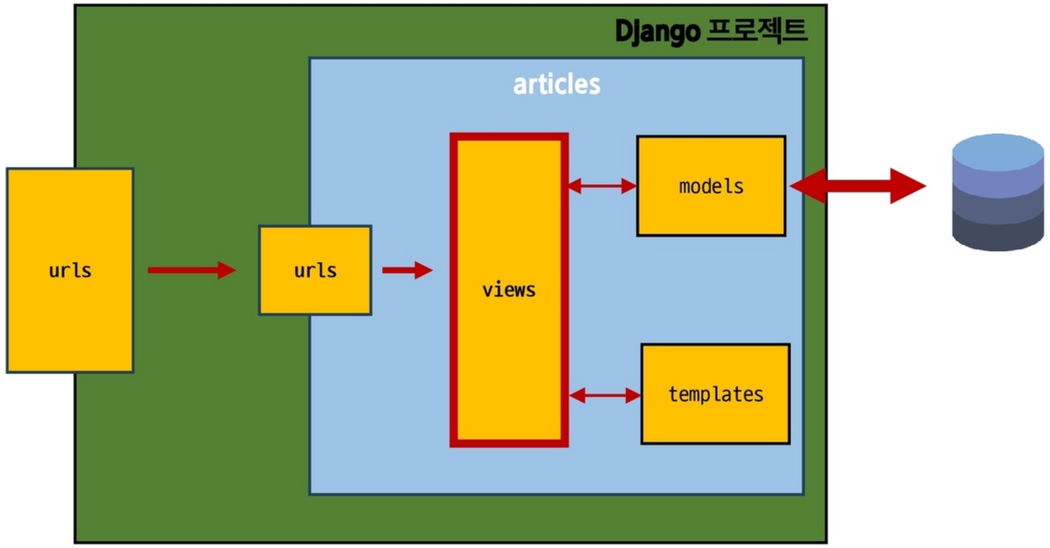
함께가요 미래로!
Enabling People

ORM with view

전체 게시글 조회

ORM with view

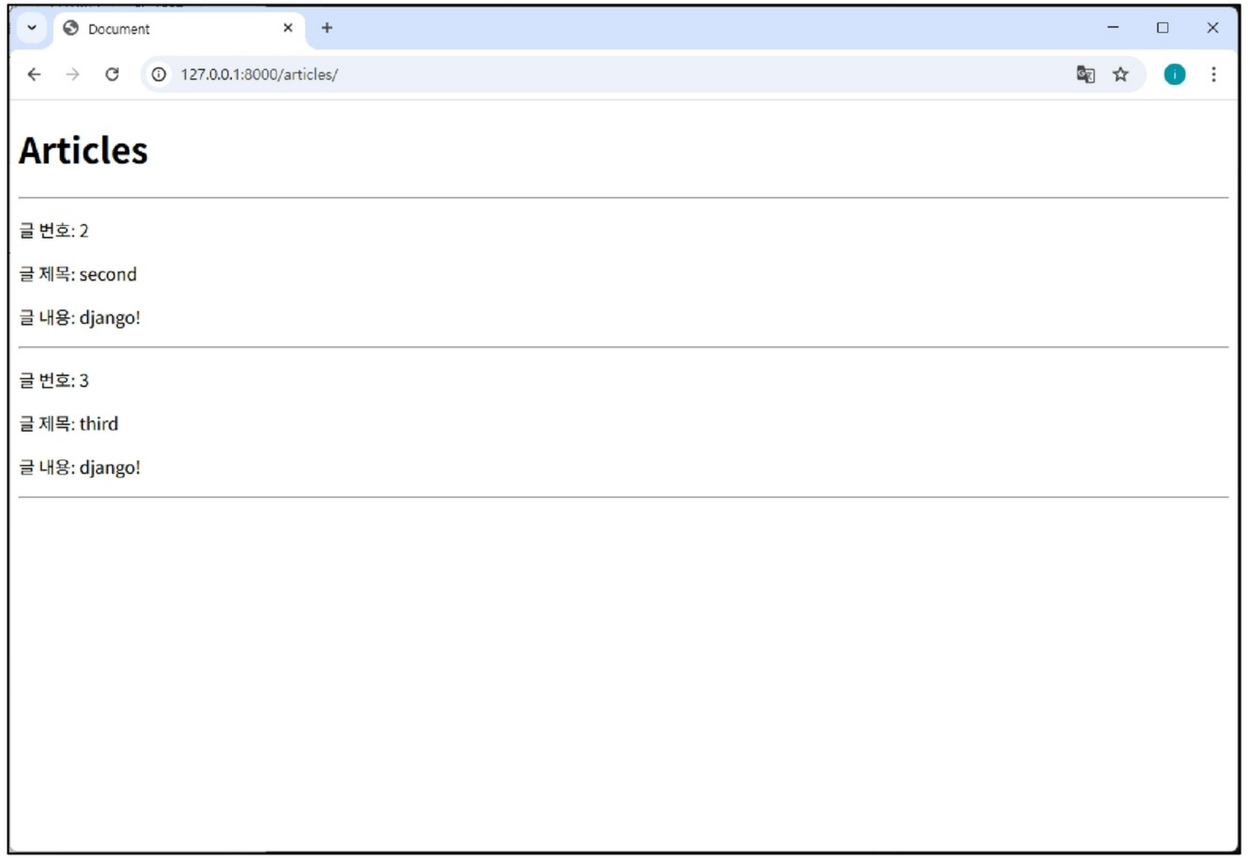
- View 함수에서 QuerySet API 활용하기
  - View에서의 QuerySet API:
    - 웹 페이지에 보여줄 데이터를 DB에서 가져올 때 사용함
    - 사용자가 입력한 새로운 데이터를 DB에 저장할 때 사용함
      - 2가지 Read(조회)
        1. **전체 게시글 조회**
        2. 단일 게시글 조회



<그림5_Django_ORM_프로젝트 구조>

전체 게시글 조회 (1/4)

- 최종 결과화면 미리보기



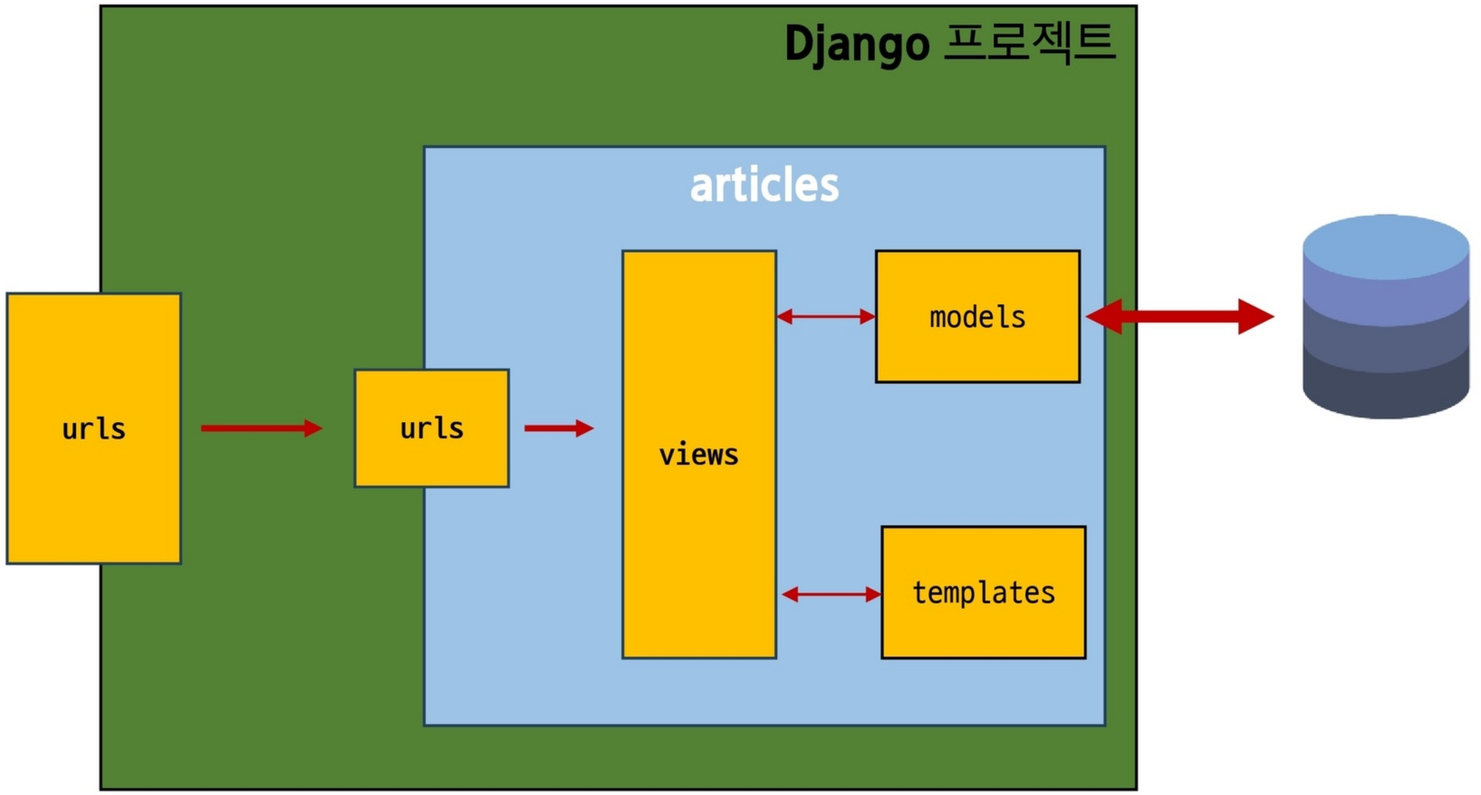
<그림6_Django_ORM_전체 게시글 조회 결과 예시>

전체 게시글 조회 (2/4)

- 요청 정의

requests

http://127.0.0.1:8000/articles/



<그림7_Django_ORM_프로젝트 구조 with 요청>

전체 게시글 조회 (3/4)

- 요청 정의

```python
# crud/urls.py
from django.urls import path, include

urlpatterns = [
    path('admin/', admin.site.urls),
    path('articles/', include('articles.urls')),
]
```

```python
# articles/urls.py
from django.urls import path
from . import views

urlpatterns = [
    path('', views.index, name='index'),
]
```

※ crud 는 Django 프로젝트 이름

TIP

앱 URL에서 빈 문자열('')의 의미란?

- 프로젝트 urls.py에서 include("articles.urls")는 articles/로 시작하는 모든 요청을 articles 앱의 urls.py로 넘겨줍니다.이때, articles/ 부분은 잘려나가고 나머지 URL만 앱의 urls.py로 전달됩니다.
- 따라서, **path('', views.index)는 articles/ 바로 뒤에 아무것도 없는 경우(articles/ 자체)**를 의미하여, 해당 view 함수를 실행하게 됩니다.

전체 게시글 조회 (4/4)

```python
# articles/views.py

from .models import Article

def index(request):
  articles = Article.objects.all()
  context = {
    'articles': articles,
  }

  return render(request, 'articles/index.html', context)
```

```
<!-- templates/articles/index.html -->

<h1>Articles</h1>
<hr>
{% for article in articles %}
  <div>
    <p>글 번호: {{ article.pk }}</p>
    <p>글 제목: {{ article.title }}</p>
    <p>글 내용: {{ article.content }}</p>
  </div>
  <hr>
{% endfor %}
```

참고

Field lookups

Field lookups: 데이터 필터링의 마법

Field lookups

단순 동치 비교(=)를 넘어 더 상세한 조건으로 데이터를 조회할 수 있도록 Django ORM이 제공하는 기능

- 예를 들어, '**특정 단어가 포함**된 제목', '**특정 날짜 이후**에 작성된 글' 등을 찾을 수 있게 해줍니다.
- 이를 통해 복잡한 데이터 조회도 파이썬 코드로 간결하고 강력하게 처리할 수 있습니다.

https://docs.djangoproject.com/en/5.2/ref/models/querysets/#field-lookups

Field lookups의 간단한 예시

- `title` 필드가 'second'로 시작하는 `Article` 데이터(레코드)를 모두 찾고 싶다면?

```
Article.objects.filter(title__startswith='second')
```

모델 필드명 **조회 조건** **비교할 값**

- Field Lookups은 모델의 필드 이름 뒤에 이중 밑줄(double underscore, __)을 붙이고, 원하는 조회 유형을 명시하는 방식으로 사용
- `filter()`, `exclude()` 및 `get()` 에 대한 키워드 인자로 지정, 손쉽게 필터링 로직을 구성

```
# Field lookups 예시
# " 내용에 'dja'가 포함된 모든 게시글 조회"
Article.objects.filter(content__contains='dja')

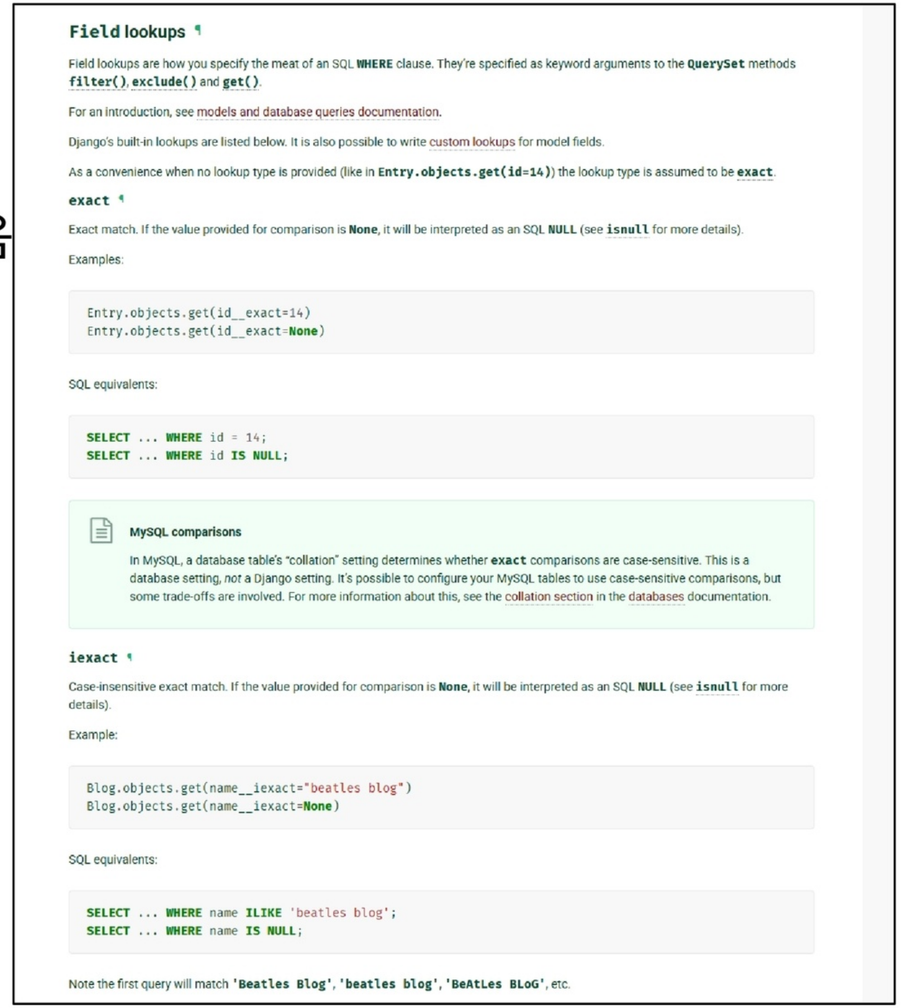
# "제목이 he로 시작하는 모든 게시글 조회"
Article.objects.filter(title__startswith='he')
```

다양한 조건의 Field lookups 조회 조건

- **exact / iexact**
  - **exact**: 대소문자를 구분하여 정확히 일치하는 값을 찾음
  - **iexact**: 대소문자 구분 없이(대소문자 무시) 정확히 일치하는 값을 찾음
- **contains / icontains**
  - **contains**: 문자열 내에 특정 값이 **포함**되어 있는지 (대소문자 구분)
  - **icontains**: 문자열 포함 여부를 대소문자 구분 없이 확인
- **비교 연산자 (gt, gte, lt, lte)**
  - 숫자 또는 날짜 필드에 대해 **크거나 작음**을 비교

※ 우리가 사용하는 SQLite에서는 `contains`도 대소문자를 구분하지 않습니다. (DB 종류에 따라 결과가 달라질 수 있음)

※ 더 다양한 조회 조건의 종류와 사용 방법은 공식 문서를 참고



Field lookups ¶

Field lookups are how you specify the meat of an SQL **WHERE** clause. They're specified as keyword arguments to the **QuerySet** methods **filter()**, **exclude()** and **get()**.

For an introduction, see models and database queries documentation.

Django's built-in lookups are listed below. It is also possible to write custom lookups for model fields.

As a convenience when no lookup type is provided (like in **Entry.objects.get(id=14)**) the lookup type is assumed to be **exact**.

exact ¶

Exact match. If the value provided for comparison is **None**, it will be interpreted as an SQL **NULL** (see **isnull** for more details).

Examples:

```
Entry.objects.get(id__exact=14)
Entry.objects.get(id__exact=None)
```

SQL equivalents:

```
SELECT ... WHERE id = 14;
SELECT ... WHERE id IS NULL;
```

MySQL comparisons

In MySQL, a database table's "collation" setting determines whether **exact** comparisons are case-sensitive. This is a database setting, *not* a Django setting. It's possible to configure your MySQL tables to use case-sensitive comparisons, but some trade-offs are involved. For more information about this, see the collation section in the databases documentation.

iexact ¶

Case-insensitive exact match. If the value provided for comparison is **None**, it will be interpreted as an SQL **NULL** (see **isnull** for more details).

Example:

```
Blog.objects.get(name__iexact="beatles blog")
Blog.objects.get(name__iexact=None)
```

SQL equivalents:

```
SELECT ... WHERE name ILIKE 'beatles blog';
SELECT ... WHERE name IS NULL;
```

Note the first query will match **'Beatles Blog'**, **'beatles blog'**, **'BeAtLes BLoG'**, etc.

https://docs.djangoproject.com/en/5.2/ref/models/querysets/#field-lookups

SAMSUNG SW AI ACADEMY FOR YOUTH

함께가요 미래로!
Enabling People

ORM, QuerySet API를 사용하는 이유

ORM, QuerySet API를 사용하는 이유는?

1. 데이터베이스 추상화
   - 개발자는 특정 데이터베이스 시스템에 종속되지 않고 일관된 방식으로 데이터를 다룰 수 있음
2. 생산성 향상
   - 복잡한 SQL 쿼리를 직접 작성하는 대신 Python 코드로 데이터베이스 작업을 수행할 수 있음
3. 객체 지향적 접근
   - 데이터베이스 테이블을 Python 객체로 다룰 수 있어 객체 지향 프로그래밍의 이점을 활용할 수 있음

실습

ORM

- 3028. todo 프로젝트 설정과 데이터 조회 (with Shell)
- 3029. todo 데이터 조회, 생성, 수정 (with Shell)
- 3030. todo 데이터 삭제와 모델 수정 (with Shell)

Test

확인 문제

Q 각 문제에 올바른 답안을 생각해봅시다

1 Django에서 ORM(Object Relational Mapping)의 주된 목적은 무엇인가?

a) HTML 렌더링을 단순화하기 위해
b) SQL 쿼리를 직접 작성하지 않고 데이터베이스를 조작하기 위해
c) 서버의 속도를 향상시키기 위해
d) CSS 파일을 자동으로 생성하기 위해

2 다음 중 Django ORM으로 새로운 게시글 객체를 생성하는 코드로 알맞은 것은?

a) Post.new(title="New Post")
b) Post.make(title="New Post")
c) Post.objects.create(title="New Post")
d) Post.objects.add(title="New Post")

3 모든 게시글을 조회하기 위한 Django ORM 코드는?

a) Post.all()
b) Post.objects()
c) Post.objects.get()
d) Post.objects.all()

4 게시글 ID가 1인 객체의 제목을 수정하는 올바른 코드 조합은?

a) Post.objects.set(id=1, title="수정됨")
b) Post.objects.get(id=1).update(title="수정됨")
c) Post.update(id=1, title="수정됨")
d)
```
post = Post.objects.get(id=1)
post.title="수정됨"
post.save()
```

확인 문제

Q 각 문제에 올바른 답안을 생각해봅시다

5 게시글 하나를 삭제하는 가장 알맞은 방법은?

a) Post.objects.delete(id=1)
b) Post.objects.get(id=1).delete()
c) Post.remove(id=1)
d) delete Post where id=1

6 제목이 정확히 "Welcome"인 게시글만 조회하고 싶을 때 알맞은 쿼리는?

a) Post.objects.filter(title="Welcome")
b) Post.objects.filter(title__contains="Welcome")
c) Post.objects.get(title__startswith="Welcome")
d) Post.objects.exclude(title="Welcome")

7 제목에 "Django"라는 단어가 포함된 게시글을 조회하는 코드는?

a) Post.objects.filter(title__has="Django")
b) Post.objects.filter(title_contains="Django")
c) Post.objects.filter(title__contains="Django")
d) Post.objects.get(title__contains="Django")

8 Django ORM을 사용하는 주요 장점 중 옳지 않은 것은?

a) 코드의 가독성이 높아진다
b) SQL Injection의 위험을 줄인다
c) 모든 쿼리를 수동으로 작성해야 한다
d) 다양한 데이터베이스에 대해 추상화된 접근이 가능하다

확인 문제

Q 각 문제에 올바른 답안을 생각해봅시다

9 작성자가 'anonymous'인 게시글을 모두 삭제하는 가장 적절한 방법은?

a) Post.delete(author="anonymous")
b) Post.objects.filter(author="anonymous").delete()
c) Post.remove(author="anonymous")
d) Post.objects.get(author="anonymous").delete_all()

10 다음 중 게시글 제목을 일괄 변경할 때 사용할 수 있는 QuerySet API는?

a) update()
b) set()
c) save_all()
d) apply()

11 전체 게시글을 삭제하려면 어떤 코드를 사용할까?

a) Post.delete()
b) Post.remove_all()
c) Post.objects.all().delete()
d) Post.objects.clear()

12 다음 중 `Post.objects.all()`과 `Post.objects.get(id=1)`의 리턴값에 대한 설명으로 옳은 것은?

a) 둘 다 단일 객체를 반환한다
b) 둘 다 QuerySet을 반환한다
c) all()은 QuerySet을 반환하고, get()은 단일 객체를 반환한다
d) get()은 리스트를 반환하고, all()은 None을 반환한다

확인 문제

정답 및 해설

1. b) SQL 쿼리를 직접 작성하지 않고 데이터베이스를 조작하기 위해 2. c) Post.objects.create(title="New Post") 3. d) Post.objects.all() 4. d) post = Post.objects.get(id=1) post.title="수정됨" post.save() 5. b) Post.objects.get(id=1).delete() 6. a) Post.objects.filter(title="Welcome") 7. c) Post.objects.filter(title__contains="Django") 8. c) 모든 쿼리를 수동으로 작성해야 한다 9. b) Post.objects.filter(author="anonymous").delete() 10. a) update() 11. c) Post.objects.all().delete() 12. c) all()은 QuerySet을 반환하고, get()은 단일 객체를 반환한다

1. Django ORM은 파이썬 코드만으로 데이터베이스를 다룰 수 있게 해주며, SQL을 몰라도 복잡한 데이터 조작이 가능하게 한다.
2. Post.objects.create()`는 새 레코드를 생성하고 저장하는 데 사용된다.
3. Post.objects.all()은 해당 모델의 모든 객체를 반환한다.
4. 객체를 조회하고 필드를 수정한 후 .save()로 반영하는 것이 일반적인 방법이다.
5. 객체를 get()으로 조회한 다음 .delete() 메서드를 호출해 삭제할 수 있다.
6. 조건 없이 문자열과 정확히 일치할 때는 필드명만 써도 되며, title="Welcome"은 title이 정확히 "Welcome"인 값을 찾는다.
7. field__contains는 해당 필드에 특정 문자열이 포함된 객체를 조회할 수 있다.
8. ORM의 주요 이점 중 하나는 쿼리를 자동으로 생성해주는 기능으로, 직접 작성할 필요가 줄어든다.
9. .filter()로 조건에 맞는 객체들을 선택하고, `.delete()`를 호출하면 해당 객체들이 모두 삭제된다.
10. update() 메서드는 필터링된 QuerySet에 대해 필드 값을 일괄적으로 수정할 때 사용된다.
11. Post.objects.all()은 전체 객체를 선택하고, .delete()는 이를 삭제한다.
12. all()은 여러 객체가 담긴 QuerySet을 반환하고, get()은 조건에 일치하는 하나의 객체만 반환한다. get() 결과는 QuerySet이 아닌 모델 인스턴스다.

활동 정리

핵심키워드

개념	설명	예시
Django Shell	프로젝트 코드 안에서 ORM 명령을 바로 실행-테스트할 수 있는 특수 파이썬 환경	python manage.py shell 또는 python manage.py shell -v 2
객체 생성 ①	**빈 인스턴스** 생성 → 필드 값 할당 → save() 호출	article = Article(); article.title='first'; article.save()
객체 생성 ②	**초기 값** 포함 인스턴스 생성 → save() 호출	article = Article(title='second', content='django!'); article.save()
객체 생성 ③	create() 단축 메서드로 **생성+저장**을 한 번에 수행	Article.objects.create(title='third', content='django!')
save()	인스턴스의 변경 사항을 데이터베이스에 **저장**하는 메서드	설명 : “객체를 DB에 저장하는 인스턴스 메서드”
all()	테이블의 **모든 레코드**를 조회하는 QuerySet 메서드	Article.objects.all() → <QuerySet […]>
filter()	조건에 맞는 **다중 레코드** QuerySet 반환	Article.objects.filter(content='django!')

활동 정리

오늘 공부한 내용 요약 및 정리

- Django Shell에서 Article 모델로 직접 CRUD 흐름을 실행하며 ORM 동작 원리를 확인했다.
- 객체 생성은 세 가지 방법으로 가능함을 익혔다.
  1. 빈 인스턴스 → 필드 값 입력 → save()
  2. 필드 값을 포함해 인스턴스 생성 → save()
  3. Article.objects.create() 단축 메서드
- 조회는 all()로 전체, filter()로 조건 다중, get()으로 조건 단일 레코드를 반환하며, get()은 반환 건수가 0건 또는 2건 이상일 때 예외를 발생시킨다.
- 수정은 조회된 인스턴스의 필드 변경 후 save()를 호출해 반영하며, 삭제는 delete()로 수행한다.
- QuerySet API를 사용하면 Python 문법만으로 데이터 추가·검색·변경·삭제가 가능하고, SQL을 직접 작성할 필요가 없다.

활동 정리

Django는 Python 문법만으로 데이터베이스와 대화할 수 있다.

- Django ORM 첫걸음

1. Django **ORM**은 파이썬과 데이터베이스 사이의 **통역사 역할**을 하며, 선언적 코드만으로 데이터 저장·조회 명령을 실행한다.
2. **QuerySet API**로 게시글을 **생성·조회·수정·삭제**하면서 데이터베이스 명령어를 직접 작성하지 않아도 된다.

```
article = Article(title='second', content='django!')
article.save() # 저장

article = Article.objects.get(pk=1) # 조회
```

SAMSUNG SW AI ACADEMY FOR YOUTH

함께가요 미래로!
Enabling People



감사합니다

